

如何正确 Vibe Coding

随着 AI 的快速发展，越来越多的同学开始用 AI 来辅助学习和工作。

但是，部分同学都用得稀烂，AI 在他手里仿佛只是一个“应付工具”，而不是一个真正的生产力工具。

我算是一个 AI 工具的重度用户。从非常早期就开始使用各种 AI 工具。比如 GitHub 的 Copilot，我是最早一批用户。OpenAI 发布的 ChatGPT 3.0 的时候，我也是第一批用的人。我甚至还给它写过一条 telegram 机器人。

这三年，我一路看着 AI 起飞。但我们同学使用 AI 的能力，却依旧原地踏步。

但我丝毫不觉得奇怪。这种状况并不是大学教育的问题，而是基础教育阶段埋下的一颗巨雷。这是 AI 的出现提前引爆了而已。

在视频的最后，我们再来讨论这个问题。

现在让我们进入正题，讲讲如何正确使用 AI。具体而言，就是如何有效地进行 vibe coding。

首先，我们有一个重要的前提，那就是打铁的各位自身得够硬。

你可以不了解具体的技术细节。但你一定要对任务有清晰理解，有能力判断——结果到底对不对。

的确，评价一个冰箱的好坏，确实不需要会制冷。但是至少你不能说出：60 摄氏度是保存新鲜水果的最佳温度。这样荒谬的话。

当你对这个领域有了一个基本的理解，甚至已经是这个领域的专家之后，那剩下的就是我们今天要讲的五个技巧了。相对而言，都很简单，可以有效提升 Vibe Coding 的成功率，快来试试吧。

第一个技巧最简单不过，就是花钱。

尤其是要做生产力用途的时候。不要犹豫，该花钱就花钱。尽量用能力最强的模型。

很多同学会说：“免费模型也挺好啊，性能至少有 80% 吧？”

问题在于——解决问题这件事是非线性的。

假设一个问题需要 100 分能力才能解决。这不意味着 80 分模型就能解决问题的 80%。

因为，对于一个问题，最终只有两种状态，解决或者未解决。

搞不定就是搞不定，你贴再多的报错信息。模型能力不够，就是两眼一抹瞎。

第二个技巧，就是打样。

写提示词 或者说 prompt，其实是个非常难的事情。它对抽象能力、总结能力要求很高。

一方面，面对一个并不太熟悉的领域，本身很难写出优秀的提示词。

另一方面，对于大部分问题，可能只需临时处理一两次，完全没有必要去花精力思考如何写出一个完美的提示词。

这个时候，我们可以偷个懒。直接给 AI 一个示例，让它照着模仿即可。

比如写报告，如果不给任何提示词，任其发挥。AI 写出的文字一眼就能看出来，那味可太足了。

但这并不意味着 AI 不能写出符合我们要求的文字。我们只需要给它一个示例，让它模仿这个示例的风格和结构。往往就能够得到一个质量不错的结果了。

写代码也是一个道理，大部分有的时候，AI 输出的代码会过于复杂，包括冗余安全检查，或是不必要的功能回退。如果，我们只是要进行原型算法验证，这样的代码反而是累赘，用起来很不方便。

这时候，这时候，比起写提示词。直接给 AI 一个我们自己以前写的代码作为示例，就要方便得多，输出的质量也基本满足需求。

第三个技巧，则是备份。八十万对六十万，机枪阵地左移五米，还是输了的教训仍在眼前。再强的 AI 也可能犯错，作为微操达人的你，词不达意的情况也时有发生。

更可怕的是，一个指令下去，往往有几百行的代码被改动，涉及到十几个相关文件。

你几乎没有办法手动撤回，尽管你可以让 AI 撤回。但根据我的经验，极大概率会越改越乱，越改越错。

这时候，一个历史备份就可以避免我们另起炉灶，把干过的活又重做一遍了。不仅如此，良好的版本控制，有助于我们后续人工排查，进一步优化代码。现在很多 Vibe Coding 工具都集成了 Git。

在 AI 时代，花半天时间学习一下 Git 的基本使用方法，绝对物超所值。

都已经用上 Vibe Coding 了，基本上就意味着我们大概率不会去仔细检查代码细节了。

但这时候一方面又想要保证代码的清晰结构和可维护性，另一方面又想尽可能把控宏观的设计思路。

这时候，就可以让 AI 在编写代码的同时，生成对应的文档。让它：• 解释模块设计 • 写接口说明 • 输出架构图 • 给出设计思路

这样，我们可以直接通过文档来把控设计思路，避免 Vibe Coding 出的代码彻底成为一个黑盒。

最后一个技巧，也是最重要的技巧，几乎决定性地影响了 AI 的使用效果。说实话——现在第一梯队的 AI，能力已经远超一般本科生甚至硕士生。

但很多同学仍反馈，AI 不够好用，难以正确复现算法代码。尤其是涉及专业知识的领域方面的编程。

据我观察，大部分情况下不是 AI 不行。而是用法不对。

说来惭愧，我作为一个专门研究矩阵的人，还写过「矩阵之美（算法篇）」。

直到现在，写 MATLAB 代码的时候，还经常出现矩阵乘法维度不匹配，这样的低级错误。我很少能一次就写出正确的代码，往往需要迭代好几轮。

人是这样，AI 也是如此。我们不能指望 AI 一次就能写出完美的代码。我们需要给它反馈，告诉它哪里不对，哪里需要改进。更重要的是，我们要让这个反馈过程自动化，形成一个闭环，让 AI 能够不断地修正自己的错误，逐步提升代码的质量。

有的时候，我需要复现一个算法，我并不单单让 AI 写一个函数就完事了。我会让它同步生成一个性能测试代码，比如，测试不同信噪比下，算法的估计精度。与此同时，我还会让它绘制出性能曲线图，并保存曲线数据到一个 CSV 文件里。然后，告诉 AI，这个算法的性能曲线应该是什么样的趋势。让 AI 根据这个反馈来修正代码，直到它能够正确地复现出预期的性能曲线为止。

掌握了以上五个技巧，本科阶段乃至硕士研究生阶段的所有编程任务，基本上都不成问题了。无论是课程设计，还是毕业论文，甚至是科研项目，都可以通过这些技巧来高效且高质量地完成。欢迎大家在评论区继续分享自己使用 AI 的经验和技巧，互相学习，共同进步。

最后，关于同学们使用 AI 的能力如此欠缺的原因，我来讲讲自己的一点观察。大家都是九年义务教育出来的，应该很清楚，我们的基础教育，本质上是一种 PVP 模式。

从小学到高中，甚至大学的很多课程，都是以考试为导向的。无论你多努力，你的极限就是卷面总分。

这一方面，会导致内卷，逼迫同学们反复进行低质量练习，提高熟练度；另一方面，在这种情况下，大家会自然而然地形成一种竞争意识。导致大多数同学不愿意主动合作，不愿意分享资源与经验，以防止他人超越自己。甚至有部分同学一有机会就想踩别人一脚，以获得相对优势。

这在过去并不算一个大问题。在几十年的发展阶段里，我们国家最缺少的是——一线技术人员（说白了就是牛马）。

并且过去的大学教育相对宽松，在一定程度上并没有过度强调竞争，甚至有些时候还鼓励合作。这为大家提供了一个从基础教育 PVP 模式过渡到实际社会生产的 PVE 模式的缓冲期。

然而，随着社会的快速发展，大学教育也逐渐“高中化”，竞争意识日益强化。这使得一直在 PVP 模式下成长起来的同学，很难突然适应真实社会生产中的 PVE 模式。

AI 的出现，更是提前引爆了这个问题。因为使用 AI，本质上就是一种合作过程。我们必须与 AI 进行清晰、有效的沟通，才能让它发挥出最大能力。

而我们，在自己的“小阁楼”里待得太久。以至于，很多人连一句完整的话都说不清楚，更别说与 AI 进行高质量沟通。

最常见的表现，就是喜欢自己造词。比如频率压缩、能量折叠、自适应结构之类的说法。这些词听起来煞有其事，实际上却没有明确的定义。这和“意大利面拌 42 号混凝土”有什么区别？

更可怕的是，很多人根本意识不到自己在胡言乱语。

在这种状态下，怎么可能用好 AI 呢？我们的教育大体是出了问题。